

TRAFFIC MONITORING AND FLOW ANALYSIS IN SOFTWARE DEFINED NETWORK

21CSE456T / SOFTWARE DEFINED NETWORKS

CASE STUDY PROJECT REPORT

Submitted by

S KRITHIKA (RA2311003020416)

VIVEK RAO S M (RA2311003020409)

ARVINDH (RA2311003020432)

Under the guidance of

Dr. R. SUJEETHA, M.E., Ph.D.

(Assistant Professor, Department of Computer Science and Engineering)

VIII SEMESTER / IV YEAR

COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING AND TECHNOLOGY



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM, CHENNAI

JULY 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this mini project report titled "Traffic monitoring and flow analysis in software defined networks" is the bonafide work of "[**S KRITHIKA (RA2311003020416), VIVEK RAO S M (RA2311003020409), ARVINDH (RA2311003020432)**]" who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. R. SUJEETHA, M.E., Ph.D.

Assistant Professor

Department of Computer Science and Engineering

SRM Institute of Science and Technology

Ramapuram, Chennai.

ABSTRACT

Software Defined Networking (SDN) is a modern networking paradigm that separates the network control plane from the data forwarding plane, enabling centralized, programmable, and flexible network management. Traditional networks rely on distributed control mechanisms where each networking device independently makes forwarding decisions, making configuration and traffic management complex and less adaptable. This project presents the design and implementation of an SDN-based traffic monitoring and flow analysis system using Mininet as the network emulator, Open vSwitch as the virtual switch, and the POX controller as the centralized SDN controller. The communication between the controller and the switches is established using the OpenFlow protocol, allowing dynamic flow rule installation and centralized traffic management.

The project implements a custom network topology consisting of multiple hosts connected through an OpenFlow-enabled switch. A Python-based monitoring application continuously collects flow statistics such as packet count, byte count, flow duration, and active flow entries from the switch. The collected statistics are used to monitor network traffic, evaluate flow behavior, and analyze network performance under different traffic conditions. Network connectivity and throughput are validated using standard tools such as **ping** and **iperf**, while the controller dynamically manages forwarding rules based on observed traffic patterns.

The implementation demonstrates the advantages of Software Defined Networking in simplifying network management, improving visibility into network traffic, and enabling real-time monitoring through centralized control. Compared with conventional networking approaches, the proposed SDN solution offers greater programmability, easier policy implementation, improved scalability, and enhanced flexibility for future network applications. The project also provides a practical platform for understanding SDN architecture, OpenFlow communication, and traffic analysis techniques that can be extended to support advanced applications such as network security, intrusion detection, quality of service management, and intelligent traffic engineering.

TABLE OF CONTENTS

S.No	Title	Page No.
	Abstract	
1.	Introduction	1
2.	Objectives	4
3.	Background Concepts	5
3.1	Software Defined Networking (SDN)	5
3.2	Mininet	7
3.3	OpenFlow Protocol	7
3.4	SDN Controller	8
4.	Design	9
4.1	Architecture Diagram / Network Topology	9
4.2	Flow Monitoring Mechanism	10
4.3	Benchmark Strategy	10
5.	Implementation	11
5.1	Environment Setup	11
5.2	Controller Programming	12
5.3	Execution Details	14
6.	Result and Analysis	15
6.1	Experimental Results	15
6.2	Performance Metrics	15
6.3	Comparative Table: Traditional Network vs SDN	16
6.4	Other Visualizations	16
7.	Challenges and Limitations	17
8.	Conclusion and Future Scope	18
	References	20

CHAPTER 1

INTRODUCTION

Computer networks form the backbone of modern communication systems by enabling the exchange of information between users, organizations, and devices across the globe. The rapid growth of cloud computing, Internet of Things (IoT), big data, online services, and virtualization has significantly increased the complexity of network infrastructures. As the number of connected devices continues to rise, traditional networking approaches face several challenges in terms of scalability, flexibility, traffic management, and configuration. Managing these large-scale networks requires administrators to configure each networking device individually, making the process time-consuming, error-prone, and difficult to maintain. These limitations have motivated researchers and industries to develop more efficient and programmable networking solutions.

Traditional computer networks are built on devices such as routers and switches that combine both the control plane and the data plane within the same hardware. The control plane is responsible for making routing and forwarding decisions, while the data plane is responsible for forwarding packets based on those decisions. Since every networking device independently maintains its own forwarding table and routing logic, any modification to network policies or configurations must be performed separately on each device. This distributed architecture increases management complexity, limits network flexibility, and makes it difficult to implement dynamic traffic engineering, centralized security policies, and efficient resource utilization.

Software Defined Networking (SDN) has emerged as an innovative networking paradigm that addresses these limitations by separating the control plane from the data plane. Instead of allowing each switch to make independent forwarding decisions, SDN introduces a centralized controller that manages the entire network through software. The controller maintains a global view of the network topology and communicates with forwarding devices using standardized protocols such as OpenFlow. This separation enables network administrators to program, configure, monitor, and optimize the network from a single centralized location without manually accessing every networking device.

One of the major advantages of SDN is network programmability. Since network intelligence is moved to software applications running on the controller, administrators can easily implement custom routing algorithms, traffic engineering policies, quality of service (QoS) mechanisms, security applications, and monitoring systems. The controller dynamically

installs flow rules into OpenFlow-enabled switches based on current network conditions, allowing rapid adaptation to changing traffic patterns. This centralized architecture improves network flexibility, simplifies management, reduces operational costs, and accelerates the deployment of new network services.

The OpenFlow protocol plays a crucial role in Software Defined Networking by providing a standardized communication interface between the SDN controller and network switches. Instead of relying on traditional routing protocols alone, OpenFlow allows the controller to directly manipulate the flow tables of switches. Each incoming packet is matched against predefined flow entries, and corresponding actions such as forwarding, dropping, or modifying packets are executed accordingly. If no matching rule exists, the switch forwards the packet information to the controller, which decides the appropriate action and installs new flow rules for future packets. This mechanism allows intelligent traffic management while minimizing unnecessary communication between switches and the controller after the initial flow setup.

To evaluate and experiment with SDN concepts without requiring expensive physical networking hardware, researchers commonly use **Mininet**, an open-source network emulator. Mininet enables users to create realistic virtual network topologies consisting of hosts, switches, controllers, and links on a single computer. It supports OpenFlow-enabled Open vSwitch (OVS), allowing developers to test controller applications and networking algorithms under controlled environments. Mininet provides an efficient platform for conducting network experiments, performance evaluation, protocol development, and educational demonstrations while significantly reducing infrastructure costs.

In this project, Mininet is used to build a virtual SDN environment consisting of multiple hosts connected through an OpenFlow-enabled switch. The virtual switch communicates with the **POX Controller**, which acts as the centralized control plane for the network. POX is a lightweight Python-based SDN controller widely used for research, education, and rapid prototyping of Software Defined Networking applications. The controller receives packet information from the switch, installs forwarding rules, and manages network communication dynamically based on network events. Because POX is implemented in Python, it allows developers to easily customize controller logic and develop applications for traffic monitoring, routing, and network management.

The primary objective of this project is to implement an SDN-based traffic monitoring and flow analysis system capable of collecting and analyzing network flow statistics in real time. A custom monitoring application is developed using Python to periodically retrieve flow information from the OpenFlow switch through the controller. The collected statistics include

packet count, byte count, flow duration, source and destination information, and active flow entries. These statistics help administrators understand traffic behavior, detect abnormal network activity, evaluate network performance, and optimize resource utilization.

The implemented network topology consists of one OpenFlow-enabled switch connected to multiple hosts using Mininet. Communication between hosts is verified using **ping** for connectivity testing and **iperf** for throughput measurement. During network operation, the monitoring module continuously observes flow statistics generated by different communication sessions. The controller dynamically manages forwarding rules while maintaining centralized visibility over the entire network. This approach demonstrates how SDN simplifies traffic management compared to conventional distributed networking architectures.

The implementation of this project demonstrates several advantages of Software Defined Networking, including centralized control, simplified network management, improved programmability, dynamic flow management, and enhanced visibility into network traffic. The use of Mininet and the POX controller provides a practical environment for understanding SDN architecture, OpenFlow communication, and controller-based network management without requiring specialized networking hardware. Furthermore, the project serves as a foundation for future enhancements such as intrusion detection systems, intelligent traffic engineering, machine learning-based network optimization, quality of service management, and cloud data center networking.

Overall, this project presents a practical implementation of Software Defined Networking for traffic monitoring and flow analysis using Mininet and the POX controller. By combining virtualization, centralized network control, and OpenFlow-based communication, the proposed system demonstrates how SDN can overcome the limitations of traditional networking while providing greater flexibility, scalability, and programmability. The knowledge gained through this project contributes to a deeper understanding of next-generation networking technologies and highlights the potential of SDN in building intelligent, adaptive, and efficiently managed communication networks.

CHAPTER 2

OBJECTIVES

The primary objectives of this project are as follows:

1. To design and implement a Software Defined Network (SDN) using Mininet and Open vSwitch for network virtualization.
2. To configure a POX controller for centralized network control and dynamic flow management using the OpenFlow protocol.
3. To develop a Python-based traffic monitoring application for collecting and analysing network flow statistics.
4. To monitor network traffic by measuring packet count, byte count, flow duration, and active flow entries in real time.
5. To verify network connectivity and evaluate network performance using Ping and iPerf traffic generation tools.
6. To analyse the efficiency of centralized SDN architecture in comparison with conventional network management approaches.
7. To demonstrate the advantages of Software Defined Networking, including programmability, scalability, centralized control, and simplified network management.

CHAPTER 3

BACKGROUND CONCEPTS

3.1 Software Defined Networking (SDN)

Software Defined Networking (SDN) is a modern networking architecture that separates the network control plane from the data forwarding plane. In conventional computer networks, networking devices such as routers and switches perform both packet forwarding and routing decisions independently. Each device maintains its own routing table and forwarding logic, making network management complex, time-consuming, and difficult to scale. Any change in network configuration or policy requires manual configuration of every individual device, which increases administrative overhead and the possibility of configuration errors.

The SDN architecture is generally divided into three layers:

1. Application Layer

The Application Layer contains network applications that define how the network should behave. These applications include traffic engineering, load balancing, firewall management, Quality of Service (QoS), intrusion detection systems, bandwidth management, and network monitoring. The applications communicate their requirements to the SDN controller through northbound APIs, allowing administrators to develop customized network services without modifying the underlying hardware.

2. Control Layer

The Control Layer is the core component of an SDN architecture. It consists of the SDN controller, which acts as the brain of the network. The controller collects topology information from switches, computes forwarding paths, installs flow rules, and continuously monitors network status. Since the controller has complete knowledge of the network, it can make optimized routing decisions and efficiently manage traffic. In this project, the POX Controller performs these centralized control functions.

3. Infrastructure Layer

The Infrastructure Layer, also known as the Data Plane, consists of OpenFlow-enabled switches and network devices responsible for forwarding packets. These switches do not make

independent routing decisions. Instead, they receive forwarding instructions from the SDN controller through the OpenFlow protocol. Each switch maintains a flow table containing rules that determine how packets should be processed based on header fields such as source address, destination address, protocol type, and input port.

One of the key advantages of Software Defined Networking is network programmability. Since the controller is software-based, administrators can easily implement new routing algorithms, security policies, and traffic management strategies without replacing physical networking hardware. This significantly reduces deployment time and operational costs while improving network flexibility.

Another important benefit of SDN is centralized network management. Traditional networks require administrators to configure each router and switch individually. In contrast, SDN allows the controller to apply network-wide policies through a single management interface. This reduces human errors and simplifies large-scale network administration.

Scalability is another major advantage of SDN. As organizations continue to deploy cloud computing platforms, virtual machines, and Internet of Things (IoT) devices, network infrastructures continue to grow rapidly. SDN enables efficient management of these expanding networks through centralized control and automated policy enforcement.

In addition to programmability and scalability, SDN provides enhanced network visibility. Since the controller continuously collects flow statistics from all switches, administrators can monitor network traffic in real time, identify congestion, detect abnormal communication patterns, and optimize resource utilization. These capabilities make SDN highly suitable for modern data centers, enterprise networks, cloud computing environments, and research laboratories.

In this project, Software Defined Networking serves as the foundation for implementing a centralized traffic monitoring system. The SDN controller communicates with OpenFlow-enabled switches created in Mininet, dynamically installs forwarding rules, and collects flow statistics that are used to analyse network traffic. This architecture demonstrates how SDN improves network flexibility, simplifies traffic management, and enables efficient monitoring compared to traditional networking approaches.

3.2 Mininet

Mininet is an open-source network emulator widely used for developing, testing, and evaluating Software Defined Networking (SDN) applications. It creates a realistic virtual network on a single computer by using lightweight virtualization techniques based on Linux namespaces and virtual Ethernet interfaces. Unlike traditional network simulators, Mininet executes real networking software, allowing users to run actual applications and protocols in a virtual environment. This makes it an ideal platform for SDN research, education, and rapid prototyping.

Mininet supports the creation of virtual hosts, switches, controllers, and links, enabling users to build customized network topologies without the need for expensive physical networking hardware. It also supports Open vSwitch (OVS), which acts as an OpenFlow-enabled software switch capable of communicating with SDN controllers such as POX, Ryu, ONOS, and OpenDaylight.

In this project, Mininet is used to create the virtual SDN environment. A custom topology is developed using Python, consisting of one OpenFlow switch connected to multiple hosts. The switch communicates with the POX controller through the OpenFlow protocol, while the hosts exchange traffic for monitoring and performance analysis. Mininet provides a command-line interface (CLI) through which network connectivity is verified using commands such as `pingall`, and network performance is evaluated using `iperf`.

3.3 OpenFlow Protocol

OpenFlow is the standard communication protocol used in Software Defined Networking to establish communication between the SDN controller and OpenFlow-enabled switches. It acts as the interface that enables the controller to manage the forwarding behavior of network switches dynamically. Instead of relying on traditional distributed routing protocols, OpenFlow allows the controller to directly install, modify, or remove flow entries from the switch's flow table.

Each OpenFlow switch maintains one or more flow tables. Every incoming packet is compared with the flow entries stored in these tables. If a matching rule is found, the corresponding action is executed, such as forwarding the packet to a specific port, dropping the packet, modifying packet headers, or sending it to another network device. If no matching entry exists, the switch

forwards the packet information to the controller using a Packet-In message. The controller analyses the packet, determines the forwarding decision, and installs a new flow rule into the switch using a Flow-Mod message. Future packets belonging to the same flow are then forwarded directly by the switch without consulting the controller again.

OpenFlow defines several categories of messages exchanged between the controller and switches. Controller-to-Switch messages include Flow Modification (Flow-Mod), Features Request, Configuration messages, and Statistics Request. Asynchronous messages include Packet-In, Flow Removed, and Port Status notifications, which allow switches to inform the controller about network events. Symmetric messages, such as Hello, Echo Request, Echo Reply, and Error messages, are exchanged between both devices to maintain communication and verify connectivity.

3.4 POX Controller

The POX Controller is an open-source Software Defined Networking controller developed using the Python programming language. It is widely used for education, research, and prototype development because of its lightweight architecture and ease of customization. POX provides a flexible framework for implementing SDN applications and supports the OpenFlow 1.0 protocol for communication with OpenFlow-enabled switches.

The controller acts as the central intelligence of the SDN network. It continuously maintains information about network topology, connected switches, active hosts, and installed flow rules. Whenever an OpenFlow switch encounters a packet that does not match any existing flow entry, it sends a Packet-In message to the POX controller. The controller analyses the packet, determines the forwarding path, and installs an appropriate flow rule into the switch. This process enables centralized traffic management and reduces unnecessary network overhead after the initial flow installation.

One of the key strengths of the POX controller is its modular architecture. Developers can easily create custom Python modules for routing, traffic monitoring, load balancing, firewall implementation, topology discovery, and network security. Since the controller is entirely software-based, new functionalities can be added without modifying the underlying networking hardware.

CHAPTER 4

DESIGN

4.1 Architecture Diagram / Network Topology

The proposed system is designed using the Software Defined Networking (SDN) architecture, where the control plane is separated from the data plane. The network consists of a centralized POX Controller, an Open vSwitch (OVS), and multiple hosts connected through Mininet. The controller manages the network by installing flow rules on the switch, while the switch forwards packets according to these rules. This architecture simplifies network management and provides centralized control over the entire network.

In the implemented topology, one OpenFlow-enabled switch is connected to four hosts (H1, H2, H3, and H4). The POX controller runs as a separate process and communicates with the switch using the OpenFlow 1.0 protocol. When a packet arrives at the switch without a matching flow entry, the switch sends a Packet-In message to the controller. The controller analyses the packet, determines the forwarding path, and installs the appropriate flow rule into the switch. Once the rule is installed, subsequent packets of the same flow are forwarded directly by the switch without contacting the controller again.

Network Configuration

Parameter	Value
Controller	POX Controller
Protocol	OpenFlow 1.0
Switch	Open vSwitch (OVS)
Number of Switches	1
Number of Hosts	4
Network Emulator	Mininet
Programming Language	Python

4.2 Flow Monitoring Mechanism

The primary objective of the project is to monitor network traffic by collecting flow statistics from the OpenFlow switch. The monitoring process is handled by a custom Python application running on the POX controller. The controller periodically sends flow statistics requests to the switch and receives detailed information about every active flow in the network.

The collected flow statistics include packet count, byte count, flow duration, source and destination addresses, and the number of active flow entries. These statistics are useful for understanding traffic behaviour, analysing network performance, and identifying abnormal communication patterns. The monitoring process consists of the following steps:

1. A host generates network traffic.
2. The OpenFlow switch checks its flow table.
3. If no matching rule exists, the packet information is sent to the POX controller.
4. The controller installs an appropriate flow rule into the switch.
5. The monitoring module periodically requests flow statistics from the switch.
6. The collected statistics are displayed and analysed to evaluate network performance.

4.3 Benchmark Strategy

To evaluate the effectiveness of the proposed SDN system, several experiments were conducted using standard network testing tools. Network connectivity between all hosts was verified using the ping command available in Mininet. The pingall command was used to ensure that every host could communicate successfully with every other host in the topology.

The following performance metrics were considered during the evaluation:

- Network Connectivity
- Throughput (Mbps)
- Packet Count
- Byte Count
- Flow Duration
- Active Flow Entries
- Controller Response

The experimental results obtained from these tests demonstrate that the SDN architecture provides centralized control, simplified traffic management, and efficient monitoring of network flows. Compared to traditional networks, the proposed system offers better programmability, easier configuration, and improved visibility into network traffic, making it suitable for modern networking environments.

CHAPTER 5

IMPLEMENTATION

5.1 Environment Setup

The implementation of the proposed Software Defined Networking (SDN) system was carried out in a Linux-based environment using Mininet as the network emulator and the POX controller as the centralized network controller. Mininet was selected because it provides a lightweight platform for creating realistic virtual network topologies using Linux namespaces and virtual Ethernet interfaces. The POX controller was used to manage OpenFlow switches and dynamically install flow rules during network communication. Communication between the controller and the switch was established using the OpenFlow 1.0 protocol. The development environment consisted of the following software tools:

Software	Version/Purpose
Ubuntu Linux	Operating System
Python	Controller and Monitoring Program
Mininet	Network Emulator
Open vSwitch (OVS)	Virtual OpenFlow Switch
POX Controller	SDN Controller
OpenFlow	Version 1.0 Communication Protocol
iPerf	Throughput Testing
Ping	Connectivity Testing

The custom network topology was created using a Python script. The topology consisted of one OpenFlow switch connected to four hosts. After creating the topology, the POX controller was started separately and connected to the switch. Once the connection was established, all packet forwarding decisions were controlled centrally by the POX controller.

The following commands were used during the implementation:

```
# Start POX Controller
./pox.py forwarding.l2_learning
# Start Mininet using custom topology
sudo python topo.py
# Verify Network Connectivity
mininet> pingall
# Generate TCP Traffic
mininet> iperf h1 h2
# View Flow Entries
sudo ovs-ofctl dump-flows s1
```

The successful execution of these commands confirmed that the SDN environment was functioning correctly and ready for traffic monitoring experiments.

5.2 Controller Programming

The controller is the core component of the Software Defined Networking architecture because it controls the forwarding behaviour of all OpenFlow switches. In this project, the POX controller was programmed using Python to implement centralized traffic management and network flow monitoring.

A custom monitoring module was developed to collect flow statistics from the OpenFlow switch at regular intervals. Whenever a packet arrived at the switch without an existing flow entry, the switch generated a Packet-In message and sent it to the controller. The controller analysed the packet header, determined the forwarding path, and installed an appropriate flow rule into the switch. This reduced unnecessary communication for subsequent packets belonging to the same flow. The monitoring module periodically requested flow statistics from the OpenFlow switch. The collected statistics included:

- Packet Count
- Byte Count
- Flow Duration

- Source MAC Address
- Destination MAC Address
- Input Port
- Output Port
- Active Flow Entries

These statistics were displayed in the controller terminal and were used to analyse network traffic during communication between hosts.

The implementation follows a simple sequence of operations:

1. The POX controller starts and waits for switch connections.
2. The OpenFlow switch establishes a connection with the controller.
3. Hosts begin transmitting packets.
4. The switch checks its flow table.
5. Unknown packets are forwarded to the controller.
6. The controller installs new flow rules.
7. Flow statistics are collected periodically.
8. The monitoring module displays traffic information.

The controller application was developed using Python because of its simplicity and excellent support for networking libraries. The modular architecture of POX also allowed easy integration of the traffic monitoring module without modifying the existing controller framework. A simplified algorithm for the controller is shown below.

Algorithm

- Step 1: Start the POX Controller.
- Step 2: Wait for OpenFlow switch connection.
- Step 3: Receive Packet-In message from the switch.
- Step 4: Analyse packet header information.
- Step 5: Generate forwarding decision.
- Step 6: Install flow rule into the switch.
- Step 7: Periodically request flow statistics.
- Step 8: Display packet count, byte count, and flow duration.
- Step 9: Repeat until network execution is completed.

5.3 Execution Details

The execution of the project involved several sequential steps to establish communication between the controller and the virtual network. Initially, the POX controller was launched, after which the Mininet topology was created using the custom Python topology script. The OpenFlow switch automatically established a connection with the controller through the OpenFlow protocol.

Once the topology was initialized, the pingall command was executed to verify connectivity among all hosts. Successful responses confirmed that the controller had correctly installed forwarding rules and that communication between all hosts was functioning properly.

After verifying connectivity, network traffic was generated using the iPerf utility. TCP communication was established between selected hosts to measure network throughput. During traffic transmission, the monitoring application continuously collected flow statistics from the switch and displayed them in the controller terminal.

The implementation process followed the sequence below:

Step 1: Launch the POX controller.

Step 2: Create the custom Mininet topology.

Step 3: Connect the OpenFlow switch to the controller.

Step 4: Verify connectivity using the pingall command.

Step 5: Generate traffic between hosts using iPerf.

Step 6: Monitor flow statistics through the custom Python application.

Step 7: Analyse the collected packet count, byte count, and flow duration.

Step 8: Repeat the experiment for different communication sessions to observe traffic behaviour.

The monitoring module successfully captured flow statistics for all active communication sessions. Since the controller maintained a global view of the network, it was able to manage forwarding decisions efficiently while continuously monitoring traffic. The collected data demonstrated the effectiveness of centralized network management provided by the SDN architecture.

The implementation confirmed that the proposed system successfully achieved centralized traffic monitoring, efficient flow management, and real-time network analysis using Mininet and the POX controller. The generated statistics were later used for performance evaluation and comparison with conventional networking approaches.

CHAPTER 6

RESULT AND ANALYSIS

6.1 Experimental Results

The proposed Software Defined Networking (SDN) system was successfully implemented using **Mininet**, **Open vSwitch (OVS)**, and the **POX Controller**. The custom network topology consisting of one OpenFlow switch and four hosts was created and tested in the Mininet environment. The controller successfully established communication with the switch through the OpenFlow protocol and managed packet forwarding dynamically.

Initially, the **pingall** command was executed to verify connectivity among all hosts. The results confirmed that all hosts were able to communicate successfully without packet loss. This demonstrated that the controller correctly installed flow entries in the OpenFlow switch.

To evaluate network performance, **iPerf** was used to generate TCP traffic between hosts. The throughput test showed stable data transmission with minimal delay. During the communication process, the monitoring module continuously collected flow statistics such as packet count, byte count, and flow duration from the switch. These statistics confirmed that the controller was successfully monitoring active network flows in real time.

6.2 Performance Metrics

The performance of the SDN network was evaluated using connectivity, throughput, and flow monitoring statistics. The collected results indicated that the centralized controller efficiently managed network communication and installed flow rules whenever new traffic was detected.

Metric	Observation
Network Connectivity	Successful between all hosts
Packet Loss	0%
Throughput	Stable during TCP transmission
Flow Monitoring	Successfully collected
Controller Response	Fast and consistent

The results demonstrate that the proposed SDN architecture provides efficient traffic management and real-time monitoring. The centralized controller simplifies network administration while maintaining reliable communication between hosts.

6.3 Comparison Between Traditional Network and SDN Network

The implemented SDN architecture was compared with a conventional network to understand the benefits of centralized network management. Unlike traditional networks, where each device independently performs routing decisions, the SDN controller maintains complete control over the network and dynamically manages traffic using OpenFlow.

Parameter	Traditional Network	SDN Network
Control Plane	Distributed	Centralized
Network Management	Manual	Programmable
Flow Control	Static	Dynamic
Traffic Monitoring	Limited	Real-time
Scalability	Moderate	High
Configuration	Device-by-device	Centralized

The comparison shows that SDN offers greater flexibility, easier configuration, and improved visibility into network traffic. Centralized control enables efficient flow management and simplifies network administration.

6.4 Analysis

The experimental results demonstrate that the proposed SDN system successfully performs centralized traffic monitoring and flow analysis. The POX controller efficiently managed packet forwarding and continuously collected flow statistics from the OpenFlow switch. The use of Mininet provided a cost-effective environment for testing the network without physical hardware.

Overall, the project validates the effectiveness of Software Defined Networking in improving network programmability, simplifying management, and enabling real-time traffic monitoring. The developed system can be further extended for applications such as network security, congestion detection, and intelligent traffic management.

CHAPTER 7

CHALLENGES AND LIMITATIONS

During the implementation of the Software Defined Networking (SDN) traffic monitoring system, several challenges were encountered. Although the project was successfully completed using Mininet and the POX controller, certain technical limitations affected the implementation and evaluation process. One of the major challenges was establishing proper communication between the **POX controller** and the **OpenFlow switch**. Incorrect controller configurations or network initialization errors occasionally prevented the switch from connecting to the controller. These issues were resolved by verifying the controller IP address, port number, and OpenFlow settings before starting the Mininet topology.

Another challenge was collecting flow statistics in real time. Since the controller periodically requests statistics from the switch, excessive polling may increase controller workload in larger network environments. In this project, a suitable monitoring interval was selected to balance timely data collection and controller performance.

The project was implemented using **Mininet**, which is a software-based network emulator. Although Mininet provides an excellent platform for SDN experimentation, it cannot completely replicate the behaviour of physical networking devices. Factors such as hardware processing speed, dedicated switching resources, and real network traffic conditions are not fully represented in the emulated environment.

Scalability is another limitation of the current implementation. The network topology consists of a single OpenFlow switch and four hosts. While this configuration is sufficient to demonstrate traffic monitoring and flow analysis, larger enterprise or data centre networks would require multiple switches, additional controllers, and more complex routing mechanisms.

The project also focuses primarily on **traffic monitoring and flow analysis**. Advanced SDN features such as Quality of Service (QoS), load balancing, intrusion detection, fault tolerance, and network security mechanisms were not included within the scope of this implementation.

Despite these limitations, the project successfully demonstrates the core concepts of Software Defined Networking, including centralized network control, dynamic flow management, and real-time traffic monitoring. The implemented system provides a strong foundation for future research and can be extended to support larger topologies, multiple controllers, security applications, and intelligent traffic management techniques.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project successfully designed and implemented a Software Defined Networking (SDN) based traffic monitoring and flow analysis system using Mininet, Open vSwitch (OVS), and the POX Controller. The OpenFlow protocol was used to establish communication between the controller and the virtual switch, enabling centralized network management and dynamic flow rule installation.

A custom network topology consisting of one OpenFlow switch and four hosts was created in Mininet. The POX controller efficiently managed packet forwarding and continuously monitored network traffic by collecting flow statistics such as packet count, byte count, and flow duration. Network connectivity was verified using the pingall command, while iPerf was used to generate traffic and evaluate network performance.

The experimental results demonstrated that the proposed SDN architecture provides centralized control, simplified network management, and efficient traffic monitoring. Compared to traditional networking, the SDN approach offers better programmability, improved flexibility, and easier implementation of network policies. The project also provided practical experience with SDN technologies and highlighted the advantages of separating the control plane from the data plane.

Overall, the implementation achieved its objectives by successfully monitoring network flows and analysing traffic within a virtual SDN environment. The project demonstrates the effectiveness of Software Defined Networking as a modern approach to network management and serves as a strong foundation for future research and development.

8.2 Future Scope

Although the project successfully demonstrates traffic monitoring in a Software Defined Network, several improvements can be made to enhance its capabilities in future work.

One possible extension is the integration of machine learning algorithms to analyse flow statistics and automatically detect abnormal traffic patterns or network congestion. This would enable intelligent traffic management and improve overall network performance.

The project can also be enhanced by implementing network security mechanisms, such as intrusion detection and DDoS attack mitigation, where the controller can dynamically identify malicious traffic and install flow rules to block suspicious communication.

Another improvement is to deploy the solution on physical OpenFlow-enabled switches instead of a virtual Mininet environment. This would allow evaluation of the system under real-world network conditions and provide more accurate performance measurements.

The network topology can be expanded to include multiple switches, multiple controllers, and a larger number of hosts to study scalability and controller performance in enterprise or data centre networks. Additional SDN controllers such as Ryu, ONOS, or OpenDaylight can also be explored and compared with the POX controller.

Furthermore, advanced SDN features such as Quality of Service (QoS), load balancing, fault tolerance, network virtualization, and cloud integration can be incorporated to improve the functionality of the system.

In conclusion, this project provides a solid platform for understanding Software Defined Networking concepts and offers significant opportunities for future enhancements in network management, security, and intelligent traffic analysis.

REFERENCES

- [1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, Apr. 2008.
- [2] A. Lara, A. Kolasani, and B. Ramamurthy, "Network Innovation using OpenFlow: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 1, pp. 493–512, First Quarter 2014.
- [3] D. Kreutz, F. M. V. Ramos, P. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, Jan. 2015.
- [4] B. Lantz, B. Heller, and N. McKeown, "A Network in a Laptop: Rapid Prototyping for Software-Defined Networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets)*, 2010.
- [5] H. M. Noman and M. N. Jasim, "POX Controller and Open Flow Performance Evaluation in Software Defined Networks (SDN) Using Mininet Emulator," *IOP Conference Series: Materials Science and Engineering*, vol. 881, no. 1, 2020.
- [6] H. Al-Gubouri, "Performance Analysis of Software-Defined Networking in Different Network Topologies," *Advances in Distributed Computing and Artificial Intelligence Journal*, vol. 13, 2024.
- [7] M. N. A. Sheikh, A. Islam, and M. S. Rahman, "A Qualitative and Comparative Performance Assessment of Software-Defined Networking Controllers," *Computers*, vol. 13, no. 4, Article 85, 2024.
- [8] H. Al-Ghamdi and N. Ahmed, "Comparative Performance Evaluation of Ryu and OpenDaylight SDN Controllers Using Mininet," *Alqalam Journal of Medical and Applied Sciences*, vol. 9, no. 1, pp. 64–67, 2026.
- [9] K. Kaur, J. Singh, and N. S. Ghumman, "Mininet as Software Defined Networking Testing Platform," *International Conference on Communication, Computing and Virtualization*, 2015.
- [10] L. Nacson, R. Puzis, and P. Zilberman, "Floware: Balanced Flow Monitoring in Software Defined Networks," *IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2016.